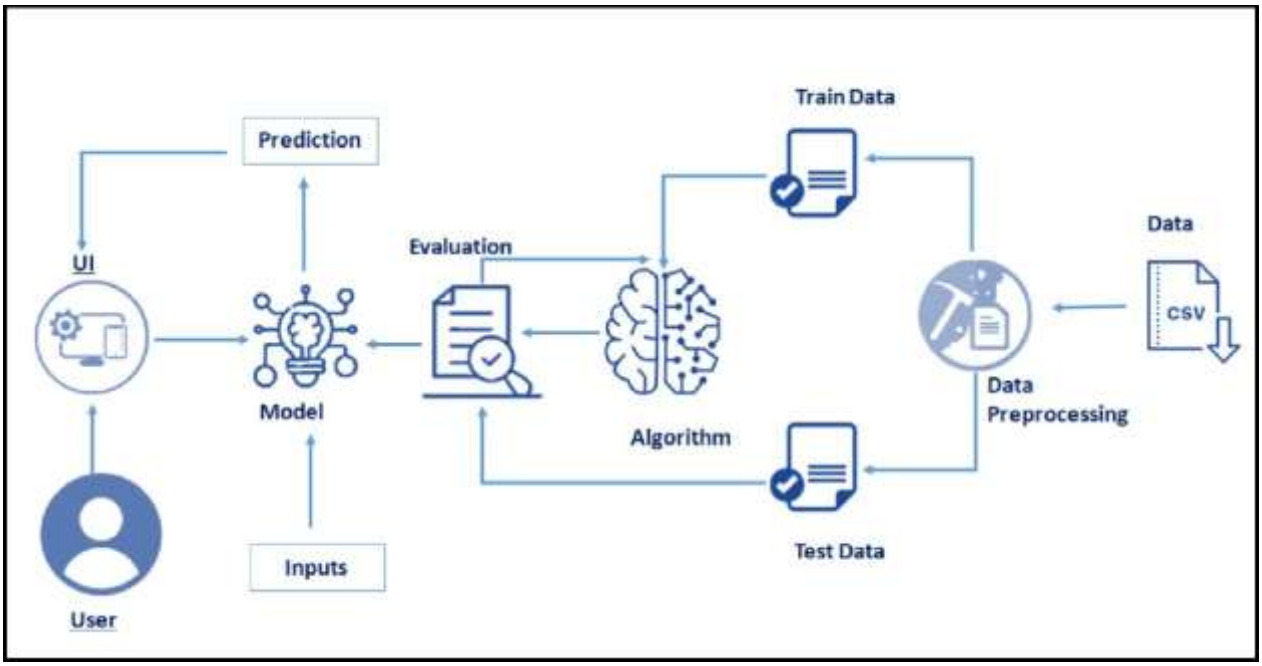
Credit Card Approval Prediction System

In today's dynamic financial landscape, applying for a credit card has become a routine step for individuals aiming to improve their financial flexibility. However, the approval process often remains opaque, leaving many applicants uncertain about their chances. Rejections without clear explanations lead to frustration and erode trust in financial systems. Factors such as age, employment duration, existing loans, and EMI commitments play a key role in determining eligibility, yet these criteria are seldom communicated transparently.

To bridge this gap, we propose a machine learning-based Credit Card Approval Prediction system. This tool accepts basic user inputs—such as age, number of employment days, existing loans, and EMI obligations—and uses a trained model to predict the likelihood of approval. The system provides a simple binary result: "approved" or "not approved," helping users assess their chances before applying. While it does not offer financial advice, it acts as a quick and effective pre-check mechanism.

This solution benefits both applicants and financial institutions. It allows users to make informed decisions, reducing unnecessary applications and the frustration of unexpected rejections. At the same time, it helps financial organizations cut down on processing time and administrative overhead. By making the credit card application process more transparent and predictable, this system promotes a smoother, more user-friendly financial experience powered by machine learning.

Technical Architecture:



Project Flow:

- User interacts with the UI to enter the input .
- Entered input is analysed by the model which is integrated.
- Once the model analyses the input the prediction is showcased on the UI

To accomplish this , we have to complete all the activities listed below:

- Define Problem / Problem Understanding
  - Specify the business problem
  - Business requirements
  - Literature Survey
  - Social or Business Impact.
- Data Collection & Preparation
  - Collect the dataset
  - Data Preparation
- Exploratory Data Analysis
  - Descriptive statistical
  - Visual Analysis
- Model Building
  - Training the model in multiple algorithms
  - Testing the model
- Performance Testing & Hyperparameter Tuning
  - Testing model with multiple evaluation metrics
  - Comparing model accuracy before & after applying hyperparameter tuning
- Model Deployment
  - Save the best model
  - Integrate with Web Framework
- Project Demonstration & Documentation
  - Record explanation Video for project end to end solution
  - Project Documentation-Step by step project development procedure

Prior Knowledge:

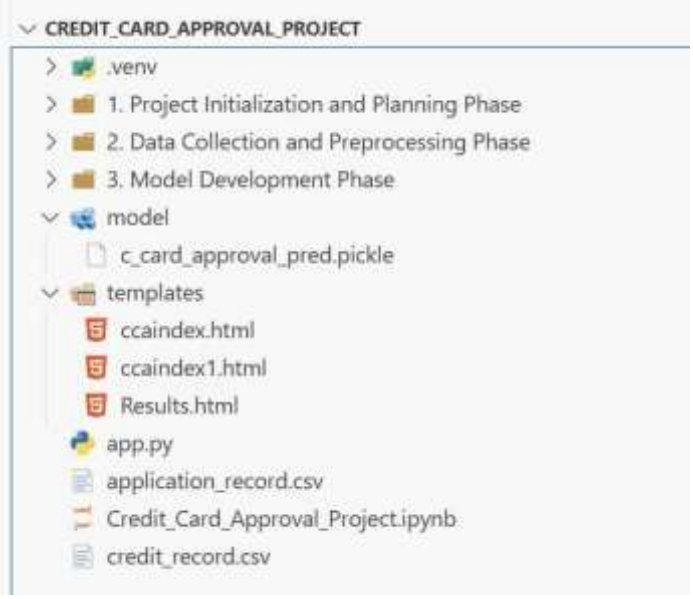
You must have prior knowledge of following topics to complete this project. ML Concepts

- Supervised learning: https://www.javatpoint.com/supervised-machine-learning
- Unsupervised learning: https://www.javatpoint.com/unsupervised-machine-learning
- Decision tree: https://www.javatpoint.com/machine-learning-decision-tree-classificationalgorithm
- Random forest: https://www.javatpoint.com/machine-learning-random-forest-algorithm
- KNN: https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning
- Xgboost: https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide- tounderstand-the-math-behind-xgboost/

- Evaluation metrics: https://www.analyticsvidhya.com/blog/2019/08/11-importantmodel-evaluation-error-metrics/

Flask Basics : https://www.youtube.com/watch?v=lj4I_CvBnt0

Project Structure:



Milestone 1: Define Problem/ Problem Understanding

In today's fast-evolving financial ecosystem, credit card applications are processed based on complex and often opaque eligibility criteria. Applicants are frequently left in the dark about the status of their approval, leading to confusion, uncertainty, and dissatisfaction. Financial institutions evaluate factors like age, employment duration, existing loans, and EMI commitments, but do not transparently communicate how these factors affect approval decisions.

This lack of clarity results in:

- Increased number of rejected applications,
- Erosion of user trust in financial services,
- Wasted time and resources for both applicants and financial institutions.

To address these challenges, the goal of this project is to develop a machine learning-based Credit Card Approval Prediction System that:

- Accepts user inputs on key financial and demographic attributes,
- Predicts the likelihood of credit card approval using trained classification models,
- Displays a simple and intuitive result: “Eligible” or “Not Eligible,”
- Provides a user-friendly interface integrated with a trained ML model for real-time feedback.

This system empowers applicants to make informed decisions before submitting a formal credit card application and assists financial institutions in reducing overhead by pre-screening unlikely candidates efficiently.

Milestone 2: Data Collection & Preparation

ML depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to download the required dataset.

Activity 1: Collect the dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.In this project we have used .csv data. This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

Link: https://www.kaggle.com/datasets/buntyshah/auto-insurance-claims-data

As the dataset is downloaded. Let us read and understand the data properly with the help of some visualisation techniques and some analysing techniques.

Activity 1.1: Importing the libraries

Import the necessary libraries as shown in the image.

```
%pip install pandas
%pip install numpy
%pip install seaborn
%pip install scikit-learn
%pip install pickle-mixin
%pip install Flask
%pip install matplotlib
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split, RandomizedSearchCV

from sklearn.preprocessing import OneHotEncoder

from sklearn.metrics import classification_report,confusion_matrix, f1_score

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import GradientBoostingClassifier

from sklearn.tree import DecisionTreeClassifier
```

Activity 1.2: Read the Dataset

Our dataset format might be in .csv, excel files, .txt, .json, etc. We can read the dataset with the help of pandas.In pandas we have a function called read_csv() to read the dataset. As a parameter we have to give the directory of the csv file.

```
csv_path = "./application_record.csv"  # update with actual file name
app = pd.read_csv(csv_path)
app.head()
```

Python

	ID	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	NAME_INCOME_TYPE	NAME_EI
0	5008804	M	Y	Y	0	427500.0	Working	
1	5008805	M	Y	Y	0	427500.0	Working	
2	5008806	M	Y	Y	0	112500.0	Working	Secor
3	5008808	F	N	Y	0	270000.0	Commercial associate	Secor
4	5008809	F	N	Y	0	270000.0	Commercial associate	Secor

```
csv_path2="./credit_record.csv"
credit=pd.read_csv(csv_path2)
credit.head()
```

Python

	ID	MONTHS_BALANCE	STATUS
0	5001711	0	X
1	5001711	-1	0
2	5001711	-2	0
3	5001711	-3	0
4	5001712	0	C

Activity 2: Data Preparation

As we have understood how the data is, let's pre-process the collected data.

The download data set is not suitable for training the machine learning model as it might have so much randomness so we need to clean the dataset properly in order to fetch good results. This activity includes the following steps.

- Handling missing values
- Handling Outliers

Activity 2.1: Handling missing values

- For checking the null values, df.isna().any( ) function is used. To sum those null values we use .sum() function. From the below image we found that there are no null values present in our dataset. So we can skip handling the missing values step.

Handling Missing Values

```
app.isnull().mean()
```

```
ID                      0.000000
CODE_GENDER             0.000000
FLAG_OWN_CAR            0.000000
FLAG_OWN_REALTY         0.000000
CNT_CHILDREN            0.000000
AMT_INCOME_TOTAL        0.000000
NAME_INCOME_TYPE        0.000000
NAME_EDUCATION_TYPE     0.000000
NAME_FAMILY_STATUS      0.000000
NAME_HOUSING_TYPE       0.000000
DAYS_BIRTH              0.000000
DAYS_EMPLOYED           0.000000
FLAG_MOBIL              0.000000
FLAG_WORK_PHONE         0.000000
FLAG_PHONE              0.000000
FLAG_EMAIL              0.000000
OCCUPATION_TYPE         0.305012
CNT_FAM_MEMBERS         0.000000
dtype: float64
```

Data preprocessing steps like dropping unwanted features, categorical encoding, feature engineering were adopted in order to clean the data of any outliers and make the data fit the model perfectly.

Dropping Unwanted Features

```
app.drop_duplicates (subset=['CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'CNT_CHILDREN', 'AMT_
```

✓ 0.0s Python

Data Cleaning And Merging

```
def data_cleansing(data):
    # Adding number of family members with number of children to get overall family members.
    #data['CNT_FAM_MEMBERS'] = data['CNT_FAM_MEMBERS'] + data['CNT_CHILDREN']

    dropped_cols = ['FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE',
                    'FLAG_EMAIL', 'OCCUPATION_TYPE', 'CNT_CHILDREN']
    data = data.drop(dropped_cols, axis = 1)

    #converting birth years and days employed to years.
    data['DAYS_BIRTH'] = np.abs(data['DAYS_BIRTH'])/365 #Absolute
    data['DAYS_EMPLOYED'] = data['DAYS_EMPLOYED']/365

    #Cleaning up categorical values to lower the count of dummy variables.
    housing_type = {'House / apartment' : 'House / apartment',
                    'With parents': 'With parents',
                    'Municipal apartment' : 'House / apartment',
                    'Rented apartment' : 'House / apartment',
                    'Office apartment' : 'House / apartment',
                    'Co-op apartment' : 'House / apartment'}
```

Milestone 3: Exploratory Data Analysis

Activity 1: Descriptive statistical

Descriptive analysis is to study the basic features of data with the statistical process. Here pandas has a worthy function called describe. With this describe function we can understand the unique, top and frequent values of categorical features. And we can find mean, std, min, max and percentile values of continuous features.

```
app.describe()
```

Python

	ID	CNT_CHILDREN	AMT_INCOME_TOTAL	DAYS_BIRTH	DAYS_EMPLOYED	FLAG_MOBIL	FLAG_WORK_PHONE	FLAG_I
count	4.385570e+05	438557.000000	4.385570e+05	438557.000000	438557.000000	438557.0	438557.000000	438557.
mean	6.022176e+06	0.427390	1.875243e+05	-15997.904649	60563.675328	1.0	0.206133	0.
std	5.716370e+05	0.724882	1.100869e+05	4185.030007	138767.799647	0.0	0.404527	0.
min	5.008804e+06	0.000000	2.610000e+04	-25201.000000	-17531.000000	1.0	0.000000	0.
25%	5.609375e+06	0.000000	1.215000e+05	-19483.000000	-3103.000000	1.0	0.000000	0.
50%	6.047745e+06	0.000000	1.607805e+05	-15630.000000	-1467.000000	1.0	0.000000	0.
75%	6.456971e+06	1.000000	2.250000e+05	-12514.000000	-371.000000	1.0	0.000000	1.
max	7.999952e+06	19.000000	6.750000e+06	-7489.000000	365243.000000	1.0	1.000000	1.

Activity 2: Visual analysis

Visual analysis is the process of using visual representations, such as charts, plots, and graphs, to explore and understand data. It is a way to quickly identify patterns, trends, and outliers in the data, which can help to gain insights and make informed decisions.

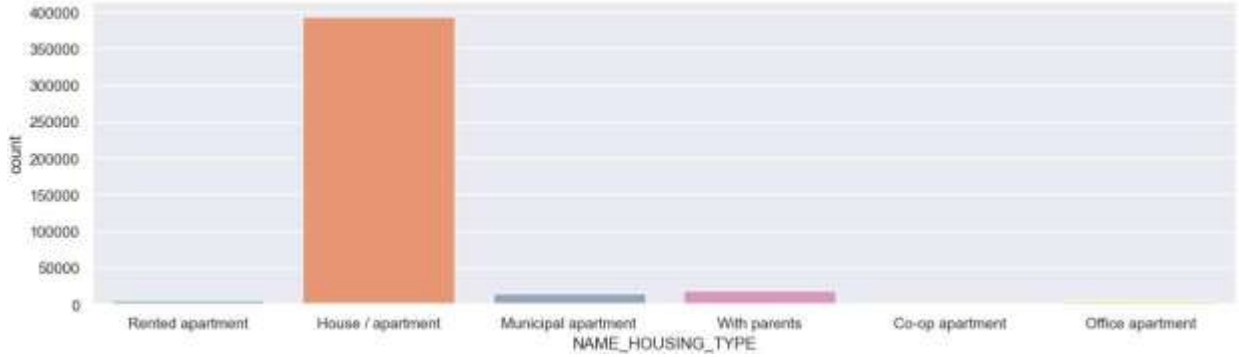
Activity 2.1: Univariate analysis

In simple words, univariate analysis is understanding the data with single feature.

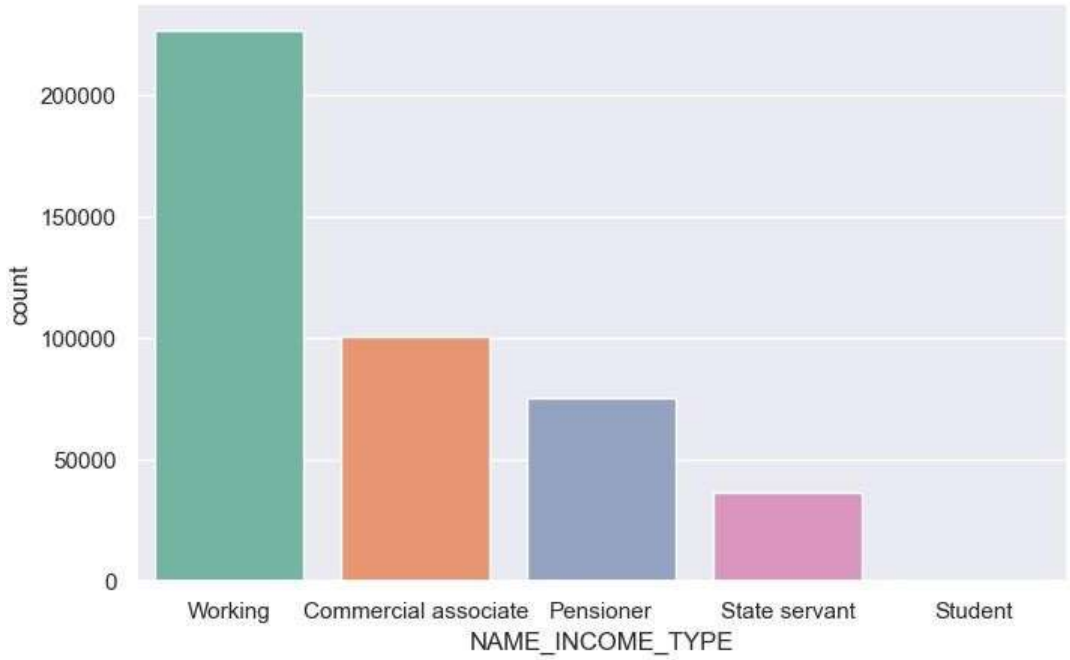
Here we have displayed two different graphs such as Piechart and countplot. Seaborn package provides a wonderful function countplot. It is more useful for categorical features.

With the help of countplot, we can Number of unique values in the feature.

Eg: Here we have counted the number of people falling under each housing type.



Eg: Here we have counted the number of people falling under a particular occupation type.



Activity 2.2: Bivariate Analysis

To find the relation between two features we use bivariate analysis.

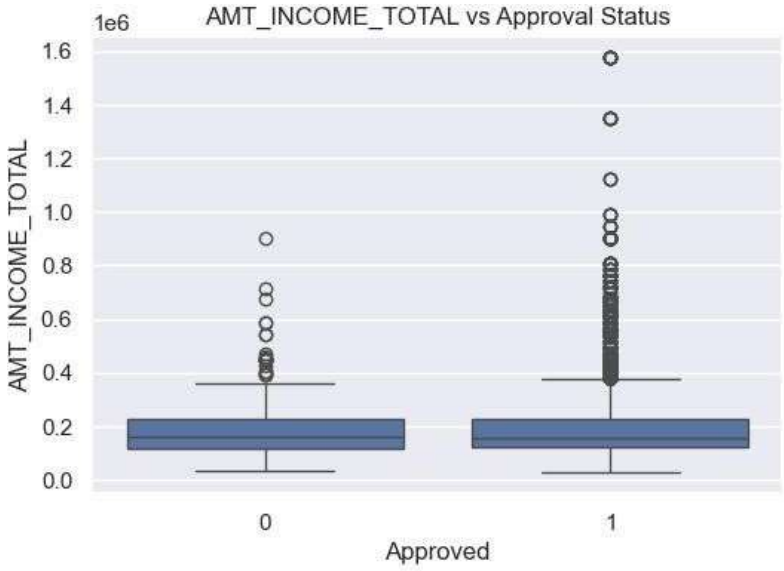
Bivariate Analysis

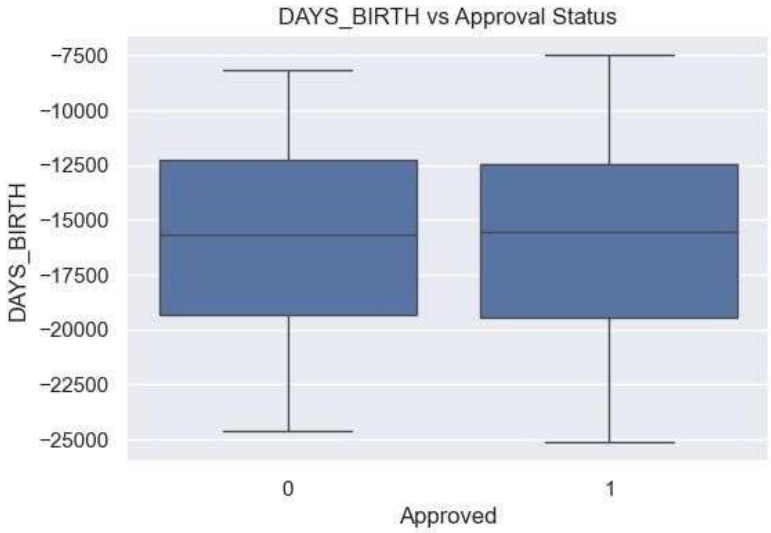
```
credit['default'] = credit['STATUS'].isin(['2', '3', '4', '5'])

# Step 2: Group by ID, mark if any default
default_status = credit.groupby('ID')['default'].max().reset_index()
default_status['Approved'] = default_status['default'].apply(lambda x: 0 if x else 1)
default_status.drop(columns='default', inplace=True)

# Step 3: Merge with application data
app = app.merge(default_status, on='ID', how='inner')
numeric = ['AMT_INCOME_TOTAL', 'CNT_CHILDREN', 'CNT_FAM_MEMBERS', 'DAYS_BIRTH', 'DAYS_EMPLOYED']

for col in numeric:
    plt.figure(figsize=(6, 4))
    sns.boxplot(x='Approved', y=col, data=app)
    plt.title(f'{col} vs Approval Status')
    plt.show()
```

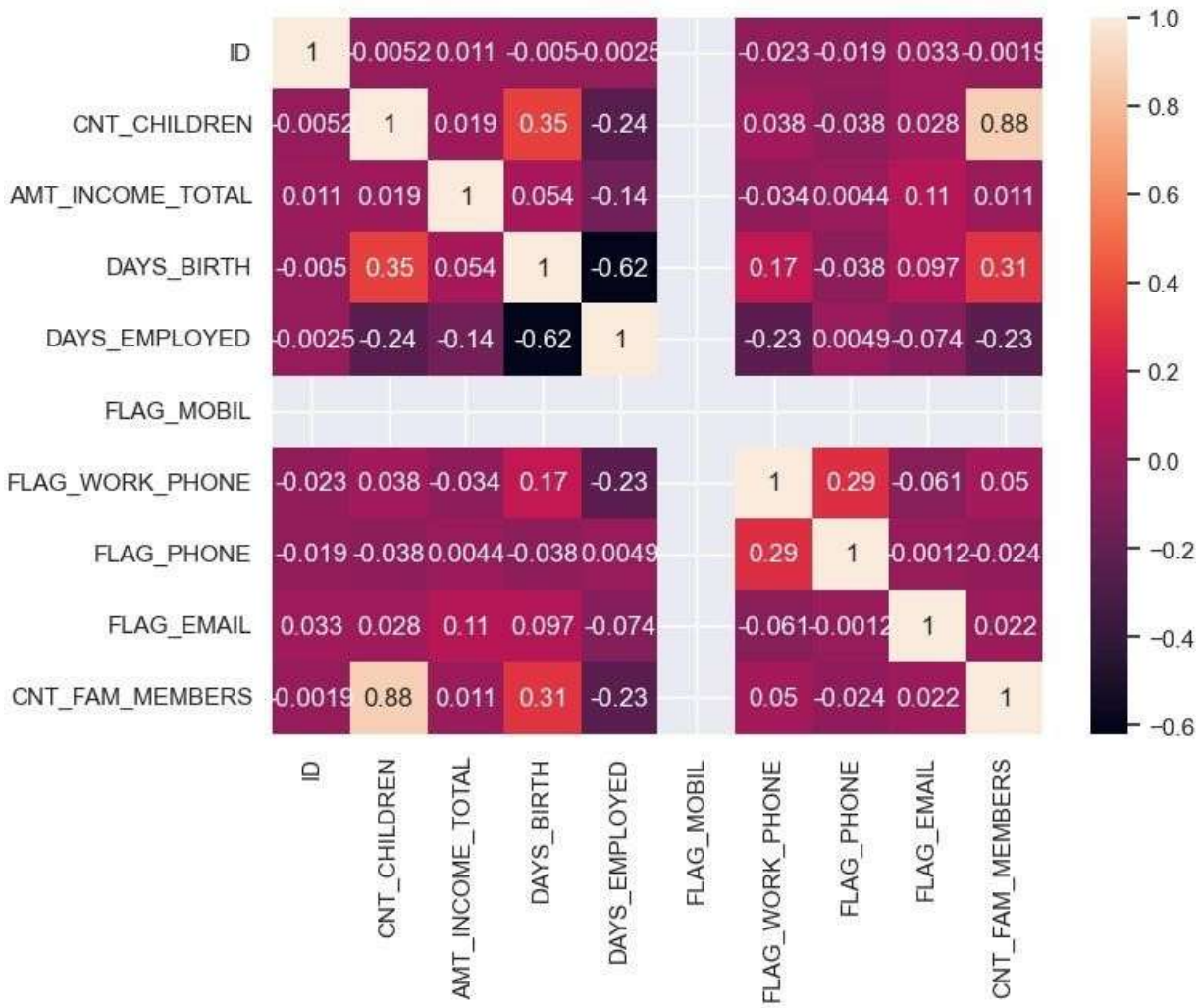




Activity 2.3: Multivariate analysis

In simple words, multivariate analysis is to find the relation between multiple features.

Here we have used heatmap from seaborn package.



Analysis:

- CNT_CHILDREN and CNT_FAM_MEMBERS have a very high correlation (**0.88**), which is expected, as family members usually include children. One of these may be redundant in predictive models.
- DAYS_BIRTH and DAYS_EMPLOYED show a notable negative correlation (**-0.62**), suggesting that younger individuals (lower absolute DAYS_BIRTH) tend to be more recently employed.
- AMT_INCOME_TOTAL shows **very weak correlation** with all other features, indicating income levels are not strongly tied to any single demographic feature in this dataset.
- Binary features like FLAG_WORK_PHONE, FLAG_PHONE, FLAG_EMAIL, and FLAG_MOBIL show very low inter-correlations (all < 0.3), suggesting they provide independent information about communication accessibility or applicant transparency.

Activity 3: Encoding The Categorical Features

• The categorical Features are can't be passed directly to the Machine Learning Model. So we convert them into Numerical data based on their order. This Technique is called Encoding.
• Here we are importing Label Encoder from the Sklearn Library.

• Here we are applying fit_transform to transform the categorical features to numerical features.

```
from sklearn.preprocessing import LabelEncoder

cg = LabelEncoder()
oc = LabelEncoder()
own_r = LabelEncoder()
it = LabelEncoder()
et = LabelEncoder()
fs = LabelEncoder()
ht = LabelEncoder()

credit_app['CODE_GENDER'] = cg.fit_transform(credit_app['CODE_GENDER'])
credit_app['FLAG_OWN_CAR'] = oc.fit_transform(credit_app['FLAG_OWN_CAR'])
credit_app['FLAG_OWN_REALTY'] = own_r.fit_transform(credit_app['FLAG_OWN_REALTY'])
credit_app['NAME_INCOME_TYPE'] = it.fit_transform(credit_app['NAME_INCOME_TYPE'])
credit_app['NAME_EDUCATION_TYPE'] = et.fit_transform(credit_app['NAME_EDUCATION_TYPE'])
credit_app['NAME_FAMILY_STATUS'] = fs.fit_transform(credit_app['NAME_FAMILY_STATUS'])
credit_app['NAME_HOUSING_TYPE'] = ht.fit_transform(credit_app['NAME_HOUSING_TYPE'])


credit_app
```

	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	AMT_INCOME_TOTAL	NAME_INCOME_TYPE	NAME_EDUCATION_TYPE
0	1	1	1	427500.0	2	1
1	1	1	1	112500.0	2	2
2	0	0	1	270000.0	2	2

Splitting data into train and test

Now let's split the Dataset into train and test sets. First split the dataset into x and y and then split the data set.Here x and y variables are created. On x variable, df is passed with dropping the target variable. And on y target variable is passed. For splitting training and testing data we are using train_test_split() function from sklearn. As parameters, we are passing x, y, test_size, random_state.

```
xtrain,xtest,ytrain,ytest=train_test_split(credit_app.drop('target',axis=1),credit_app['target'],test_size=0.2,random_state=0)
```

Milestone 4: Model Building

Activity 1: Training the model in multiple algorithms

Now our data is cleaned and it's time to build the model. We can train our data on different algorithms. For this project we are applying three classification algorithms. The best model is saved based on its performance.

Activity 1.1: Logistic Regression

A statistical model used for binary classification tasks (e.g., yes/no, 0/1).
It estimates the probability that a given input belongs to a certain class using a sigmoid function.It's simple, fast, and effective for linearly separable data.

Logistic Regression Model

```
def logistic_reg(xtrain, xtest, ytrain, ytest):
    lr = LogisticRegression(solver='liblinear')
    lr.fit(xtrain, ytrain)
    ypred = lr.predict(xtest)
    print("LogisticRegression")
    print('Confusion matrix')
    print(confusion_matrix(ytest, ypred))
    print('Classification report')
    print(classification_report(ytest, ypred))
    return lr
```

Activity 1.2: Random Forest Classifier

An ensemble learning method that builds multiple decision trees during training.
It combines the output of individual trees to make a final prediction, often by majority voting.
It improves accuracy and reduces overfitting compared to a single decision tree.

Random Forest Classifier

```
def random_forest(xtrain, xtest, ytrain, ytest):
    rf = RandomForestClassifier()
    rf.fit(xtrain, ytrain)
    ypred = rf.predict(xtest)
    print("RandomForestClassifier")
    print('Confusion matrix')
    print(confusion_matrix(ytest, ypred))
    print('Classification report')
    print(classification_report(ytest, ypred))
```

Activity 1.3: XG Boost Model

An advanced boosting algorithm that builds trees sequentially, focusing on correcting previous errors. It uses gradient boosting and regularization techniques to improve performance and prevent overfitting. Widely used in competitions for its speed and predictive power.

XG Boost Model

```
def g_boosting(xtrain, xtest, ytrain, ytest):
    gb = GradientBoostingClassifier()
    gb.fit(xtrain, ytrain)
    ypred = gb.predict(xtest)
    print("GradientBoostingClassifier")
    print('Confusion matrix')
    print(confusion_matrix(ytest, ypred))
    print('Classification report')
    print(classification_report(ytest, ypred))
```

Activity 1.4: Decision Tree Model

A flowchart-like structure where decisions are made by splitting data based on feature values. Each internal node represents a feature, branches represent decisions, and leaves represent outcomes. Easy to interpret but prone to overfitting on noisy datasets.

Decision Tree Model

```
def d_tree(xtrain, xtest, ytrain, ytest):
    dt = DecisionTreeClassifier()
    dt.fit(xtrain, ytrain)
    ypred = dt.predict(xtest)
    print("DecisionTreeClassifier")
    print('Confusion matrix')
    print(confusion_matrix(ytest, ypred))
    print('Classification report')
    print(classification_report(ytest, ypred))
```

Milestone 5: Performance Testing

Activity 1: Testing model with multiple evaluation metrics

Multiple evaluation metrics means evaluating the model's performance on a test set using different performance measures. This can provide a more comprehensive understanding of the model's strengths and weaknesses. We are using evaluation metrics for classification tasks including accuracy, precision, recall, support and F1-score.

Activity 1.1: Compare the model

For comparing the above four models, the compare_model function is defined.

```
def compare_model(xtrain,xtest,ytrain,ytest):
    logistic_reg(xtrain, xtest, ytrain, ytest)
    print('-'*100)
    random_forest(xtrain, xtest, ytrain, ytest)
    print('-'*100)
    g_boosting(xtrain, xtest, ytrain, ytest)
    print('-'*100)
    d_tree(xtrain, xtest, ytrain, ytest)
compare_model(xtrain,xtest,ytrain,ytest)
```

Performance Metrics:

Logistic Regression

```
LogisticRegression
Confusion matrix
[[1025   12]
 [   5  900]]
Classification report
              precision    recall  f1-score   support

           0       1.00      0.99      0.99      1037
           1       0.99      0.99      0.99       905

    accuracy                           0.99      1942
   macro avg       0.99      0.99      0.99      1942
weighted avg       0.99      0.99      0.99      1942
```

Random Forest Classifier

```
RandomForestClassifier
Confusion matrix
[[1022   15]
 [   8  897]]
Classification report
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      1037
           1       0.98      0.99      0.99       905

    accuracy                           0.99      1942
   macro avg       0.99      0.99      0.99      1942
weighted avg       0.99      0.99      0.99      1942
```

XG Boost Classifier

```
GradientBoostingClassifier
Confusion matrix
[[1036    1]
 [   4  901]]
Classification report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1037
           1       1.00      1.00      1.00       905

    accuracy                           1.00      1942
   macro avg       1.00      1.00      1.00      1942
weighted avg       1.00      1.00      1.00      1942
```

Decision Tree Model

```
DecisionTreeClassifier
Confusion matrix
[[1035    2]
 [   1  904]]
Classification report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1037
           1       1.00      1.00      1.00       905

    accuracy                           1.00      1942
   macro avg       1.00      1.00      1.00      1942
weighted avg       1.00      1.00      1.00      1942
```

Considering the high accuracy (>90%) hyperparameter tuning is not required for this credit card approval system.

Milestone 6: Model Deployment

Activity 1: Save the best model

Saving the best model after comparing its performance using different evaluation metrics means selecting the model with the highest performance.This can be useful in avoiding the need to retrain the model every time it is needed and also to be able to use it in the future. Here we have saved the Decision Tree Classifier since it is having the highest accuracy.

```
import pickle
import os
from sklearn.tree import DecisionTreeClassifier

# Train your model
dt = DecisionTreeClassifier()
dt.fit(xtrain, ytrain)

# Create model/ directory if it doesn't exist
os.makedirs('model', exist_ok=True)

# Save the model
with open('model/c_card_approval_pred.pickle', 'wb') as f:
    pickle.dump(dt, f)
```

Activity 2: Integrate with Web Framework

In this section, we will be building a web application that is integrated to the model we built. A UI is provided for the uses where he has to enter the values for predictions. The enter values are given to the saved model and prediction is showcased on the UI. This section has the following tasks

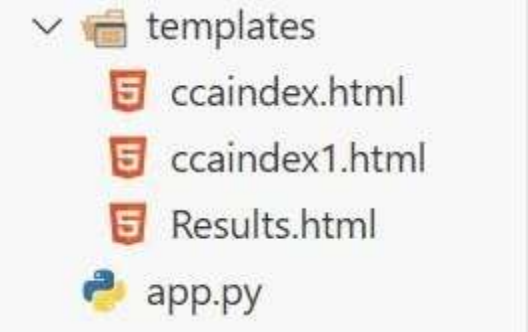
- Building HTML Pages
- Building server-side script
- Run the web application

Activity 2.1: Building Html Page:

For this project create HTML files namely

- ccaindex.html
- ccaindex1.html
- Results.html

and save them in the templates folder.



Activity 2.2: Build Python code:

Import the libraries.

```
from flask import Flask, request, render_template
import numpy as np
import pandas as pd
import pickle
import os
```

Load the saved model. Importing the flask module in the project is mandatory. An object of Flask class is our WSGI application. Flask constructor takes the name of the current module (name) as argument.

```
app = Flask(__name__)
model_path = os.path.join('model', 'c_card_approval_pred.pickle')
with open(model_path, 'rb') as handle:
    model = pickle.load(handle)
```

Render the HTML pages:

Here we will be using a declared constructor to route to the HTML page which we have created earlier. In the above example, '/' URL is bound with the ccaindex.html function. Hence, when the home page of the web server is opened in the browser, the html page will be rendered.

```
@app.route('/')
def home():
    return render_template('ccaindex.html')

@app.route('/Prediction', methods=['GET', 'POST'])
def prediction():
    return render_template('ccaindex1.html')

@app.route('/Home', methods=['GET', 'POST'])
def my_home():
    return render_template('ccaindex.html')

@app.route('/predict', methods=["POST"])
```

Whenever you enter the values from the html page the values can be retrieved using POST Method.

Retrieving the values from the UI:

```
def predict():
    try:
        # Get values from form, convert to float
        input_features = [float(x) for x in request.form.values()]
        feature_values = [np.array(input_features)]

        # Your column names (MUST match training)
        feature_names = ['CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY',
                         'AMT_INCOME_TOTAL', 'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE',
                         'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE', 'DAYS_BIRTH',
                         'DAYS_EMPLOYED', 'CNT_FAM_MEMBERS', 'paid_off',
                         '#_of_pastdues', 'no_loan']

        x = pd.DataFrame(feature_values, columns=feature_names)

        pred = model.predict(x)

        prediction = "Eligible" if pred[0] == 1 else "Not Eligible"

        return render_template("Results.html", prediction=prediction)

    except Exception as e:
        return f"Error: {e}"
```

Here we are routing our app to predict() function. This function retrieves all the values from the HTML page using Post request. That is stored in an array. This array is passed to the model.predict()

function. This function returns the prediction. And this prediction value will be rendered to the text that we have mentioned in the Results.html page earlier.

Main Function:

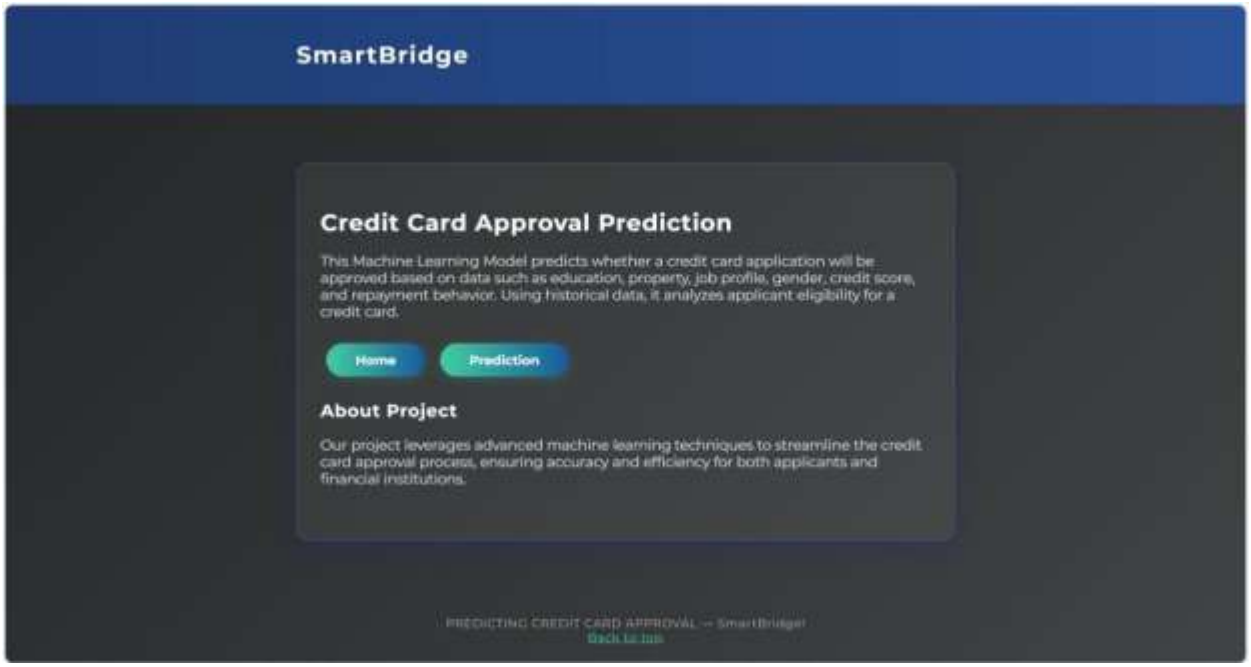
```
if __name__ == "__main__":
    app.run(debug=True)
```

Activity 2.3: Run the web application

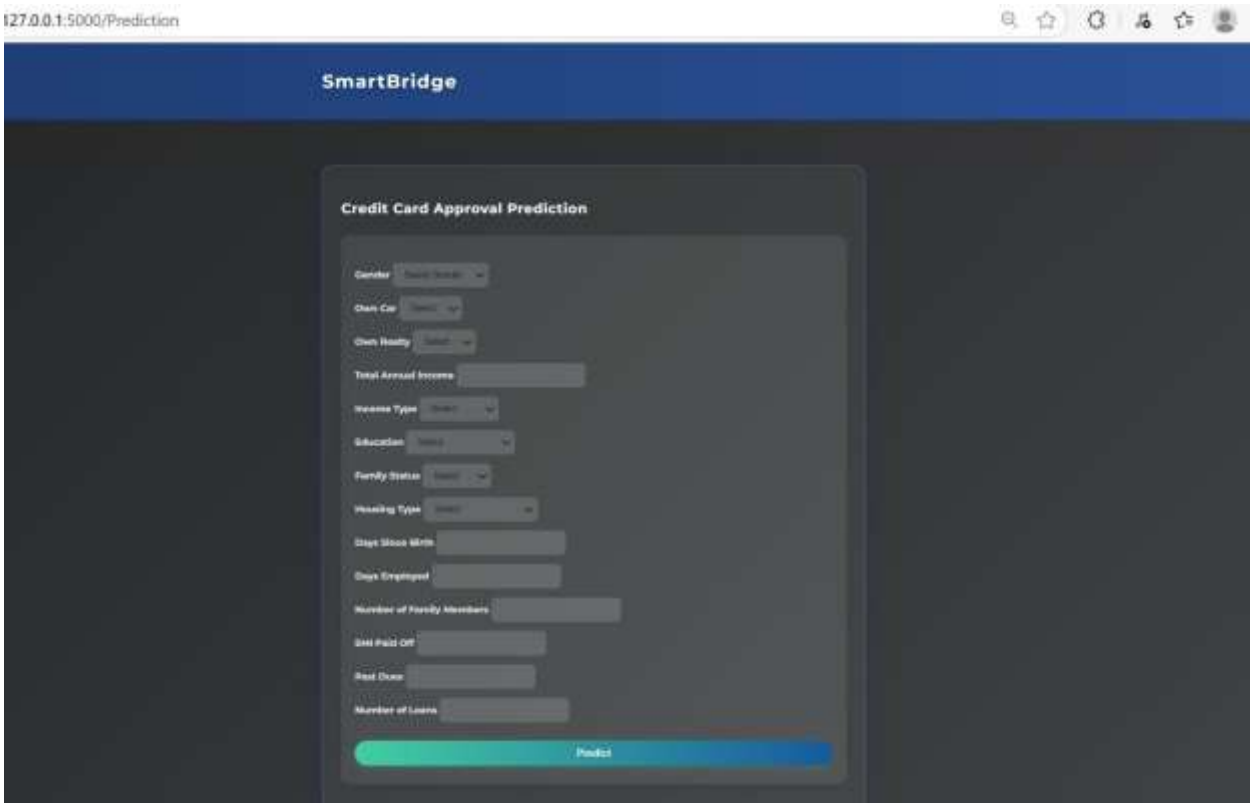
- Open the terminal (if using VS Code)
- Create a virtual environment using the command : & "-----*address of python executable environment*---- " -m
- Then activate the virtual environment using the command: .\.venv\Scripts\activate
- After activating the environment , run the file using command : python app.py
- Navigate to the localhost where you can view your web page.
- Click on the predict button on the home page, enter the inputs, click on the submit button, and see the result/prediction on the web.

```
PS D:\CREDIT_CARD_APPROVAL_PROJECT> .\.venv\Scripts\activate
(.venv) PS D:\CREDIT_CARD_APPROVAL_PROJECT> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 247-978-312
```

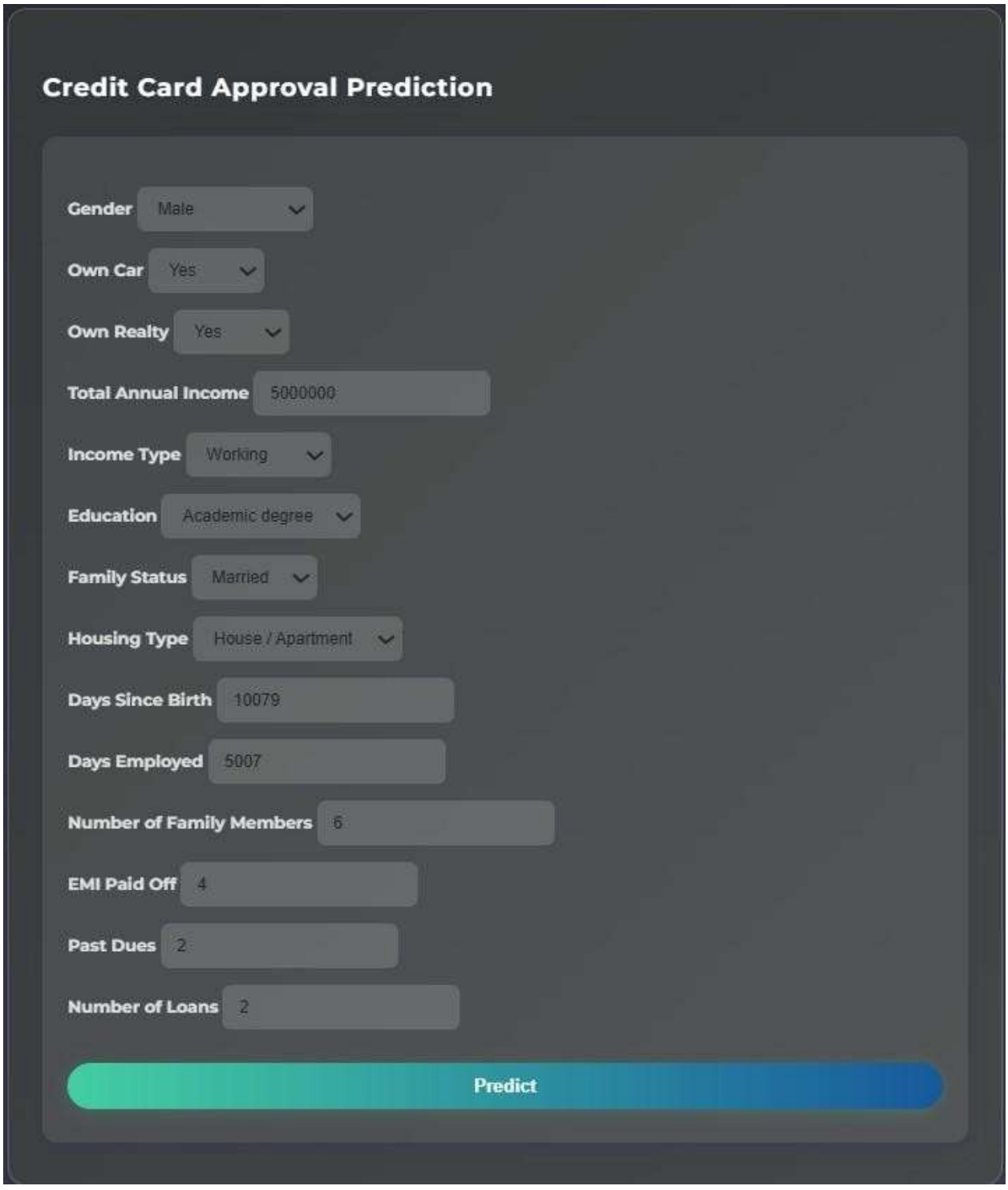
Landing Page:



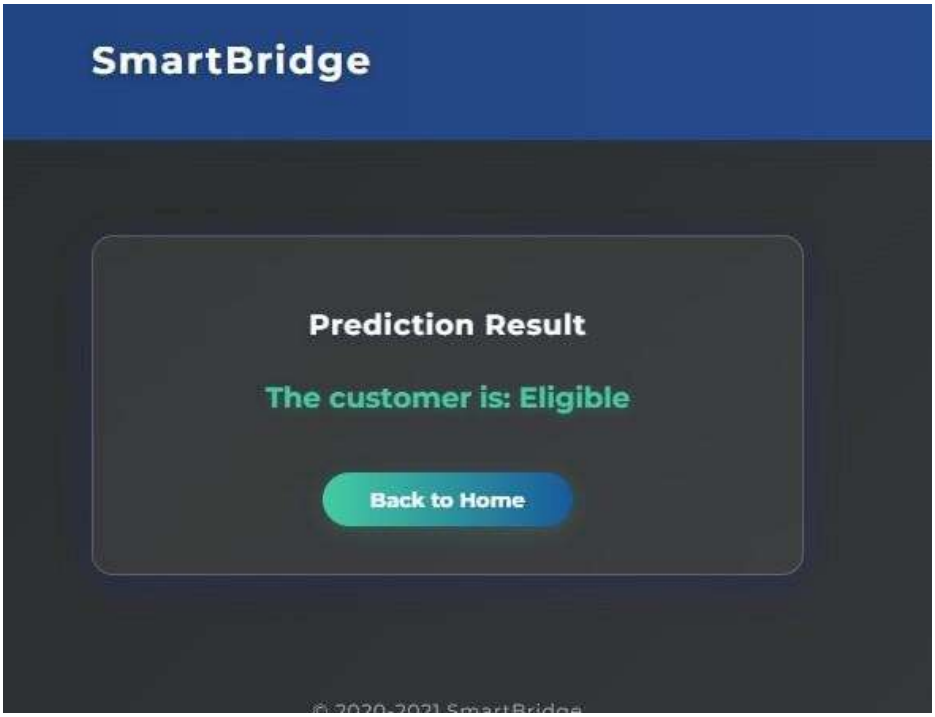
Input-Form:



Sample Input:


Credit Card Approval Prediction
Gender
Male
Own Car
Yes
Own Realty
Yes
Total Annual Income
5000000
Income Type
Working
Education
Academic degree
Family Status
Married
Housing Type
House / Apartment
Days Since Birth
10079
Days Employed
5007
Number of Family Members
6
EMI Paid Off
4
Past Dues
2
Number of Loans
2
Predict

Output:



Milestone 7: Project Demonstration & Documentation

Activity 1:- Explanation Video:

https://drive.google.com/file/d/1MxfZmVfexbzqrQWbY45XfTz_S2hXQFz_/view?usp=sh

aring

Activity 2:- Project Documentation- GitHub Repository Link:
https://github.com/Pooja17-p/credit-card-approval-prediction